# Systems programming lesson 3

## COM 312: BSc Computer science

**Topics area coverage.**

**UNIX File System**

- Linux Directory structures
- Basic shell scripting in Linux
- Unix and Posix file system
- Inodes, API, UNIX kennel support for files

## Linux Directory structures

**What's a computer File system? -** A file system is the *methods and data structures* that an operating system **uses to keep track of files on a disk or partition**; that is, the way the files are organized on the disk.

- The word is also used to refer to a partition or disk that is used to store the files or the type of the file system.

**3 types of File types**

i. **Ordinary files –** *Data, text, or program instructions.*
ii. **Special files –** *files provide access to hardware such as hard drives, CD-ROM drives, modems, and Ethernet adapters.*
iii. **Directories –** *store both special and ordinary files.*

**Linux directory structures**

- Linux keeps all the devices and data in form of files in different locations.
- It contains a root directory > sub-directories > files.
  **NB:** in Linux system you won't find drive D, E. etc only /slash or /mount point.

**Unix uses a hierarchical file system structure,** *much like an upside-down tree, with root (/) at the base of the file system and all other directories spreading from there*.

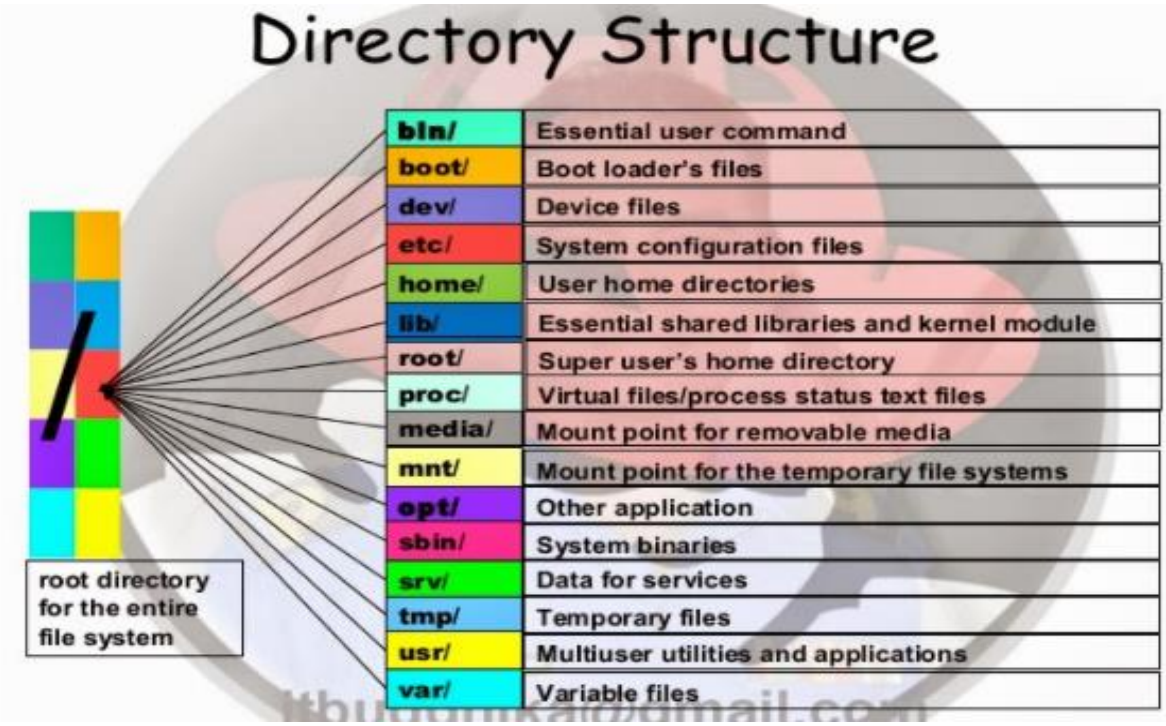
A Unix **file system** is a collection of files and directories that has the following properties

- It has a root directory (/) that contains other files and directories.
- Each file or directory is uniquely identified by its name, the directory in which it resides, and a unique identifier, typically called an **inode**.
- By convention, the root directory has an **inode** number of **2** and the **lost+found** directory has an **inode** number of **3**. **Inode numbers 0 and 1 are**

**not used.** File inode numbers can be seen by specifying the **-i option** to **ls command**.

- It is self-contained. There are no dependencies between one filesystem and another.

**the Linux file system structures and understand the meaning of individual high-level directories.**




**/bin :** All the executable binary programs (file) required during booting, repairing, files required to run into single-user-mode, and other important, basic commands viz., cat, du, df, tar, rpm, wc, history, etc.
**/boot** : Holds important files during boot-up process, including Linux Kernel.
**/dev** : Contains device files for all the hardware devices on the machine e.g., cdrom, cpu, etc
**/etc** : Contains Application's configuration files, startup, shutdown, start, stop script for every individual program.
**/home** : Home directory of the users. Every time a new user is created, a directory in the name of user is created within home directory which contains other directories like Desktop, Downloads, Documents, etc.
**/lib** : The Lib directory contains kernel modules and shared library images required to boot the system and run commands in root file system.
**/lost+found** : This Directory is installed during installation of Linux, useful for recovering files which may be broken due to unexpected shut-down.

**/media** : Temporary mount directory is created for removable devices viz., media/cdrom.
**/mnt** : Temporary mount directory for mounting file system.
**/opt** : Optional is abbreviated as opt. Contains third party application software. Viz., Java, etc.
**/proc** : A virtual and pseudo file-system which contains information about running process with a particular Process-id aka pid.
**/root** : This is the home directory of root user and should never be confused with '/'
**/run** : This directory is the only clean solution for early-runtime-dir problem.
**/sbin** : Contains binary executable programs, required by System Administrator, for Maintenance. Viz., iptables, fdisk, ifconfig, swapon, reboot, etc.
**/srv** : Service is abbreviated as 'srv'. This directory contains server specific and service related files.
**/sys** : Modern Linux distributions include a /sys directory as a virtual filesystem, which stores and allows modification of the devices connected to the system.
**/tmp** :System's Temporary Directory, Accessible by users and root. Stores temporary files for user and system, till next boot.
**/usr** : Contains executable binaries, documentation, source code, libraries for second level program.
**/var** : Stands for variable. The contents of this file is expected to grow. This directory contains log, lock, spool, mail and temp files.

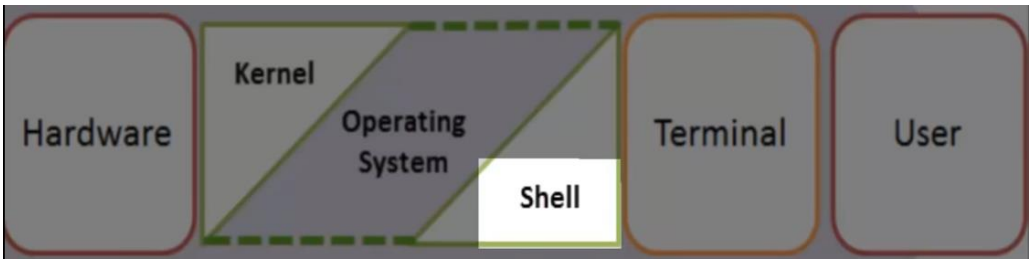
## Basic shell scripting in Linux

**Why should I learn Shell Programming at all?**

One reasons to learn shell programming. the major reason is systems administration. **It is a powerful programming method that can help you learn the command-line better,** *save time*, and do away with tedious file management tasks.

used for multiple purposes

a) such as automating repetitive tasks
b) backup and maintenance of databases,
c) monitoring a system
d) creating a programming environment,
e) linking existing programs together, and various other tasks.




- Shell is a program which interprets user commands through **CLI** like Terminal

**Shell vs Terminal**

- **Shell >** is a program that is responsible for the execution of an instruction and returning the output.
- **Terminal >** is responsible to send instructions to the shell by taking input from the user and displaying the out of the instruction to the user.

**Linux Shell**

- Linux has a kernel which talks to the hardware. But a kernel does not understand the language of a user.
  And this is where a shell comes into the picture.
- A shell is an interpreter between the kernel and the user.
- It gathers input from the users and executes the programs,

**There are various types of shell present which are listed below –**

1. Bourne shell (sh)/Bash - Bash is a Unix shell and command language written by Brian Fox for the GNU Project as a free software replacement for the Bourne shell. Its name is an acronym for **Bourne-again shell**. (Kali Linux Debian-version uses Bash shell)
2. Korn shell (ksh)
3. Bourne again shell (bash)
4. POSIX shell (sh)
5. C shell (csh)
6. TENEX/TOPS C shell (tcsh)

**Command to view different shells.**

```
root@root:~# which shell
root@root:~# cat /etc/shells
# /etc/shells: valid login shells
/bin/sh
/bin/dash
/bin/bash
/bin/rbash
/usr/bin/tmux
/usr/bin/screen
root@root:~#
```

## What is the shell scripting?

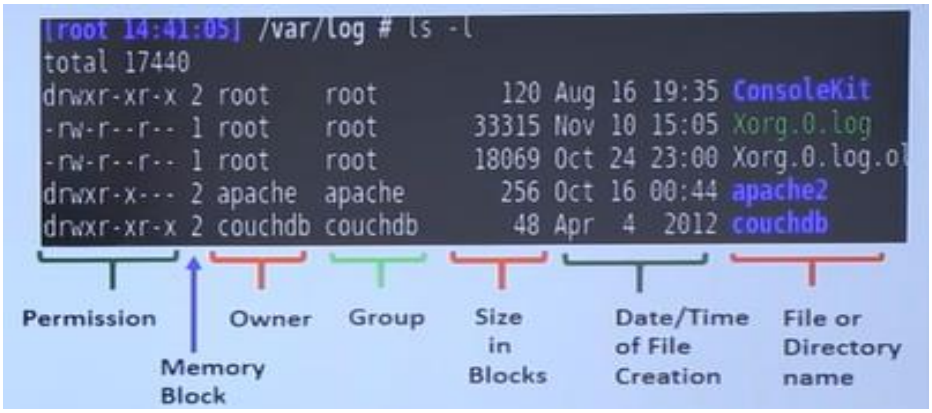
- Shell scripting is a group commands written in a sequence to automate

  **Your** :

  Regular boring stuff or Repetitive work.

- **Basic Shell Scripting Commands in Linux**: cat, more, less, head, tail, mkdir, cp, mv, rm, touch, grep, sort, wc, ls, cut and, more.

  **Let's pick ls-l (list long) shell scripting command.**




## Why bash shell?

bash shell is an advanced shell.

- **bash** is an sh- compatible shell that incorporates useful features from the korn shell(ksh) and C shell (Csh)
- Now a day bash shell is the default shell for most of the UNIX/Like systems. used for multiple purposes Example of Bash shell scripting is used to automate tasks.
  a) such as automating repetitive tasks

- ✓ install and configure httpd on RHEL & Ubuntu
- ✓ install jenkins on Tomcat servers on RHEL & Ubuntu
- ✓ find all servers FQDN and Uptime and export it to CSV file.
- ✓ Install and configure Ansible setup on AWS cloud .(Ansible Controller and Required numbers numbers of Node)

How to Automate these Tasks?

i. Step 1: get all manual steps ( commands)
ii. Step2: write a shell script
iii. Step3: now execute shell script.

The idea behind the statement above demonstrates ….installation of may be 100 servers using cloud computing. to save time system admin should write bash command using TOM cat ..to Automate the task from manual installation

**shell scripting > introduction $ she bang**

**So what's the difference** between the **shell** $ **bash** interpreter *command in Linux programming?*

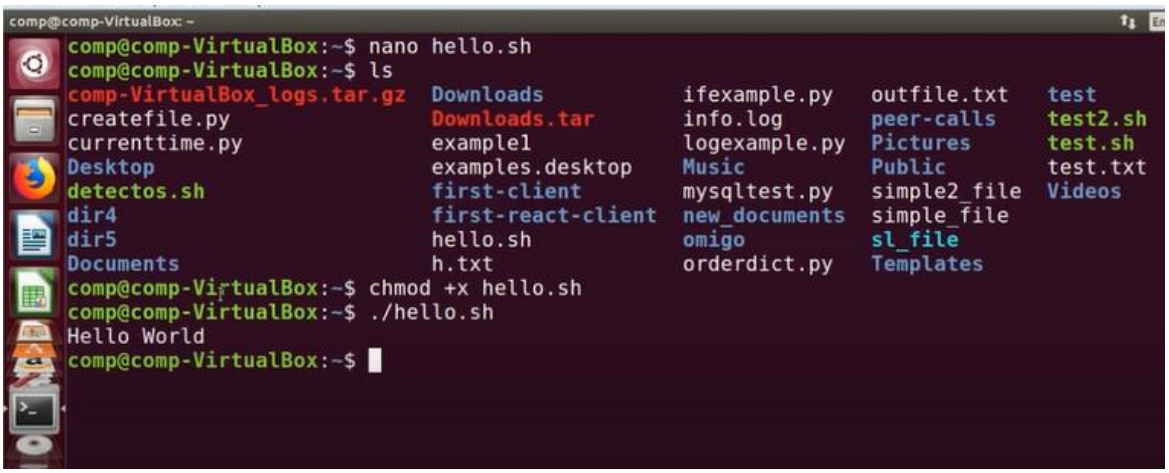
**Bash** interpreter > *offers streamlined experience. (offers full functionalities of the Unix/Linux support*)

**Shell** interpreter > *does not support full functionalities of the Unix/Linux support*)

**How to write hello world bash shell script?**

The procedure is as follows:

1. Create a new file called hello.sh using a text editor such as nano or vi:
   ```
   nano hello.sh
   ```
2. Add the following code:
   ```
   #!/bin/bash
   echo "Hello World"
   ```
3. Set the script executable permission by running chmod command:
   ```
   chmod +x hello.sh
   ```
4. Run or execute the script using following syntax:
   ```
   ./hello.sh
   ```
   OR
   ```
   sh hello.sh
   ```
   OR
   ```
   bash hello.sh
   ```




## Code explanation

- First line of the script ( `#!/bin/bash` ) is shebang. It tells the Linux/Unix how to run the script.
- **Chmode + x :** change mode

The +x parameter is **used to add the x permission** which is the symbol for the execute permission.

- The execution permission will give the execution ability to the owner user or group of the file.
  - *i.* The + *will add provided ability* and **-** *minus will remove the provided ability.*

## Bash script with user prompt and input validation

You can accept input from the keyboard and assign an input value to a user defined shell variable using read command.

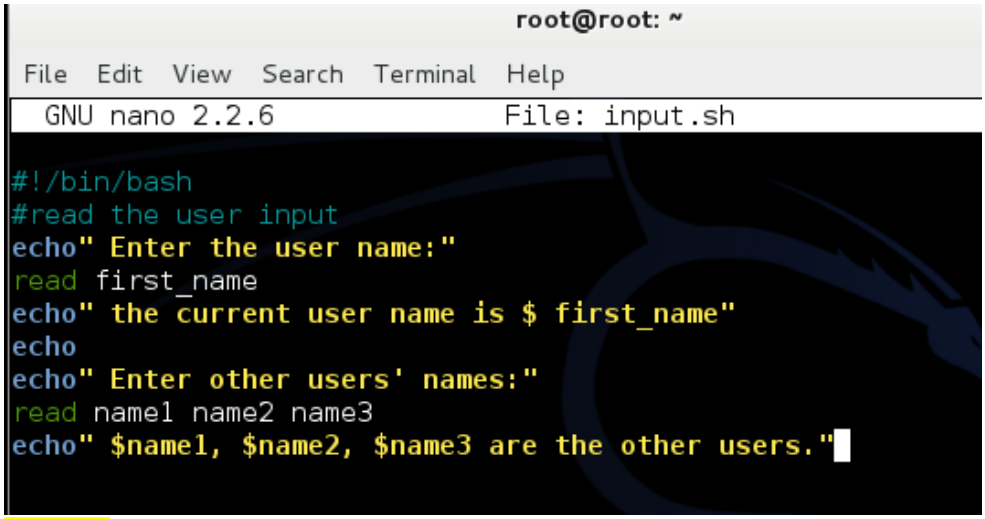
| Option | Meaning |
|---|---|
| -a ANAME | The words are assigned to sequential indexes of the array variable ANAME, starting at 0. All elements are removed from ANAME before the assignment. Other NAME arguments are ignored. |
| -d DELIM | The first character of DELIM is used to terminate the input line, rather than newline. |
| -e | **readline** is used to obtain the line. |
| -n NCHARS | **read** returns after reading NCHARS characters rather than waiting for a complete line of input. |
| -p PROMPT | Display PROMPT, without a trailing newline, before attempting to read any input. The prompt is displayed only if input is coming from a terminal. |
| -r | If this option is given, backslash does not act as an escape character. The backslash is considered to be part of the line. In particular, a backslash-newline pair may not be used as a line continuation. |

| Option | Meaning |
|---|---|
| -s | Silent mode. If input is coming from a terminal, characters are not echoed. invisible input from user i.e. take password or secret data. |
| -t TIMEOUT | Cause **read** to time out and return failure if a complete line of input is not read within TIMEOUT seconds. This option has no effect if **read** is not reading input from the terminal or from a pipe. |
| -u FD | Read input from file descriptor FD. |

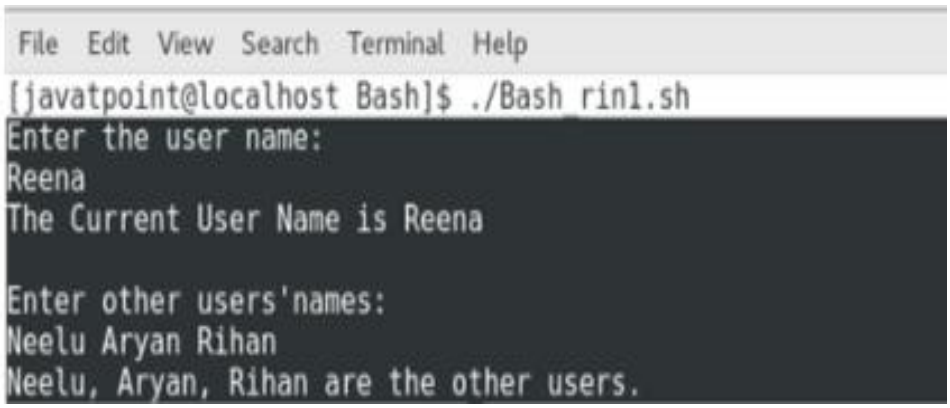
### i. Bash script with user prompt"

**Example 1**:

In this example, we read both the single and multiple variables from the Bash Script by using read command.




**output**



**Example – using simple read command**

Create a bash file with the following script to know how to use the read command with a variable. After running the script, the program will wait for the user input. When the user types the data and press enter, the data will be stored in the answer variable. The value of the answer variable will be printed later.

```
#!/bin/bash
echo -n "What is your favorite food: "
# Assign input value into a variable
read answer
# Print the value of the variable
echo "Oh! you like $answer!"
```

**Exercise**

**Run the script and give your output!!!!**

## Unix and Posix file system

There are many popular operating systems, including Windows, MacOS, Android, Chrome OS… **If each OS has its own unique standard and API**, *it would be hard to distribute the same software to multiple OS platforms.* Therefore, we need a universal standard that works across multiple OS, which is called POSIX.

# The C API

POSIX **defines its standards in terms of the C language**. Therefore, **programs are portable to other operating systems at the source code level**. Nonetheless, we can also implement it in any standardized language.

The POSIX C API adds more functions on top of the ANSI C Standard for a number of aspects:

- File operations
- Processes, threads, shared memory, and scheduling parameters
- Networking

- Memory management
- Regular expressions

## I nodes, API, UNIX kennel support for files

**Linux and other Unix-like** Operating systems **maintain consistency by treating everything as a file** (even the hardware devices). *The keyboard, mouse, printers, monitor, hard disk, processes, even the directories are treated as files in Linux*. The regular files contain data such as text (text files), music, videos (multimedia files), etc.

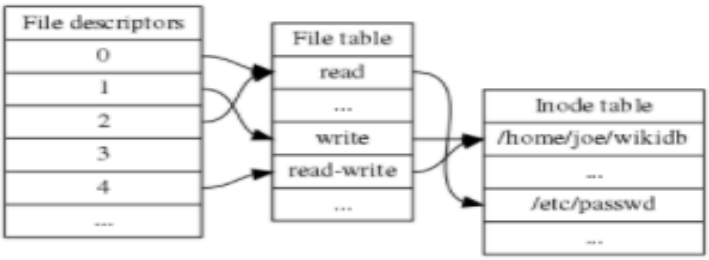
### What is an inode in Linux? (index node)

(1) An inode is a *thing* (a data structure), not a number

(2) An "inode number" is the name/address of an inode.

Every Linux file or directory has an inode, and this inode **contains all of the file's metadata.**

- **I nodes store information about** files and directories **(folders),** *such as file ownership, access mode (read, write, execute permissions),* **and file type**.




File descriptors, file table and inode table in Unix[7]

- **NB**: For better performance make your directories smaller by sorting files into subdirectories rather having one large directory.

**Why learn and use inode?**

**Inode** is a Linux and other Unix-like data structure used to keep information about a file on your server. *This includes everything on your account including files, folders, emails, code,* etc. The number of inodes indicates the number of files and folders you have. Therefore, the more files and folders you have on your server, the higher your inode usage will be.

- Keep in mind, **inode usage** differs from **disk space usage.** Disk space refers to the size of your files size (KB, MB, GB). However, inode refers to the number of files you have.
- As an example, let's suppose you are using 20GB / 30GB on your server. This will make you well within your disk space usage. However, if you have hundreds of thousands of very small files within that 20GB, the inode usage could end up negatively affecting the performance of your server.

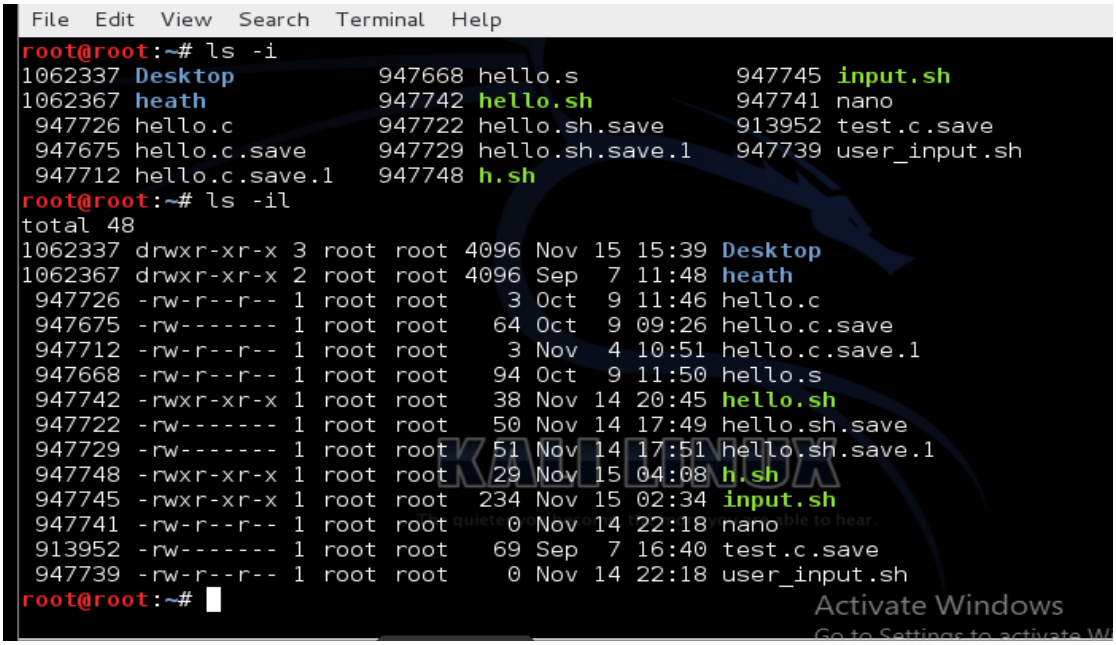
  I nodes

  • **Advantages**: – Fast access for small files (majority) – Supports large files – Supports sparse files

## What's the difference between I node number and file descriptor?

i. **I node numbers** *are like pointers within the file system*, and **are stored within the file system itself.** (**In particular, a directory entry** is the combination of a name and an inode, and you can see this using ls -i.

- An inode number always refers to something on a device somewhere




The first number on the left represents the inode number of the file.

**Within a POSIX system, a file has the following attribute contained in inode file**

- Device ID (this identifies the device containing the file; that is, the scope of uniqueness of the serial number).
- File serial numbers.

- The file *mode* which determines the file type and how the file's owner, its group, and others can access the file.
- A link count telling how many hard links point to the inode.
- The User ID of the file's owner.
- The Group ID of the file.
- The device ID of the file if it is a device file.
- The size of the file in bytes.
- Timestamps telling when the inode itself was last modified (`ctime`, *inode change time*), the file content last modified (`mtime`, *modification time*), and last accessed (`atime`, *access time*).
- The preferred I/O block size.
- The number of blocks allocated to this file.

## What can go wrong with inodes?

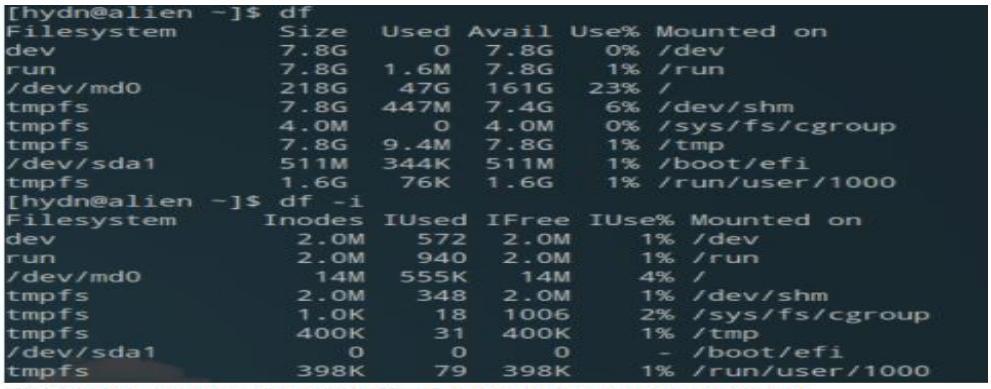
Running out of inodes can look weird, because it's likely you'll still have free disk space

**The moment a file system runs out of inodes, all new files and folders will be rejected. Until that point, no adverse effects will manifest. Once inodes are 100% used, you'll begin to notice:**

- Data loss
- Applications crashing
- OS restarting
- Processes don't restart
- Periodic tasks not firing

If you notice any of the above, use **df-i** to confirm that you've reached inode capacity

The **df command** (short for ***disk free***) is used to show the amount of free disk space available on Linux and other Unix-like systems and to understand the filesystems that have been mounted.



*df with no arguments and df -i (show free inodes) output.*

## Revision Questions

1. Write a systems program using **bash scripting** that will prompt user input while saving the input into three variables. The read statement has the keyword "-a" to take multiple values from the user and save them to the one variable array "**NAMES**". At the last echo statement, all the inputted values are printed as array members within the string text. **[5 marks]**

2. Discuss various types of files supported by Unix. Also explain the use of chmod command. **[ 5 marks]**

3. Write a Shell script to check whether the given 3 number is prime or not. [3 marks]